Q1. Load a CSV dataset into a Pandas DataFrame. Perform basic data inspection such as displaying the first few rows, checking data types, handling missing values, and removing duplicate rows.

CODE:

```
import pandas as pd

import numpy as np

df = pd.read_csv('iris.csv')


print("First 5 rows of the dataset:")

print(df.head())


print("\nDataset Information:")

print(df.info())


print("\nSummary Statistics:")

print(df.describe())


print("\nMissing Values in Each Column:")

print(df.isnull().sum())


df_cleaned = df.dropna()

print("\nNumber of duplicate rows:", df.duplicated().sum())


df_cleaned = df_cleaned.drop_duplicates()


print("\nData after cleaning:")

print(df_cleaned.info())

print(pd.options.display.max_rows)

pd.options.display.max_rows = 10
```

```
print(df_cleaned)

print(pd.options.display.max_rows)
```

Q2. Using Seaborn, create a line plot to visualize trends in a time series dataset. Customize the plot with appropriate labels, grid lines, and a title.

Code:

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt


data = {

    'Date': pd.date_range(start='2024-01-01', periods=10, freq='D'),

    'Sales': [120, 135, 150, 145, 160, 170, 180, 175, 190, 200]

}


df = pd.DataFrame(data)


plt.figure(figsize=(10, 6))


sns.lineplot(

    data=df,

    x='Date',

    y='Sales',

    marker='o'

)


plt.xlabel("Date")

plt.ylabel("Sales")

plt.title("Daily Sales Trend")
```

```
plt.grid(True)

plt.show()
```

Q3. Using Pandas, load a dataset and perform column-wise operations such as adding a new calculated column, renaming columns, changing data types, and dropping unnecessary columns.

Code:

```
import pandas as pd

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"

df = pd.read_csv(url)

df.to_csv('titanic.csv', index=False)

print("Original Data:")

print(df.head())


print(df.describe())


df['FamilySize'] = df['SibSp'] + df['Parch'] + 1


df.rename(columns={

    'SibSp': 'Siblings/Spouses',

    'Parch': 'Parents/Children',

    'Fare': 'TicketFare'

}, inplace=True)


df['Survived'] = df['Survived'].astype(bool)


# Convert 'Pclass' to category

df['Pclass'] = df['Pclass'].astype('category')
```

```
df.drop(columns=['Cabin', 'Ticket'], inplace=True)


print("\nModified DataFrame:")
print(df.head())


print("\nData Types After Modification:")
print(df.dtypes)
df.to_csv('titanicModified.csv', index=False)
```

Q4. Create a Seaborn heatmap using a correlation matrix generated from a DataFrame. Customize the color palette, annotations, and title.

Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt


data = {
    'Maths': [78, 85, 96, 80, 70],
    'Science': [88, 79, 92, 85, 76],
    'English': [90, 82, 85, 88, 80],
    'Computer': [95, 90, 98, 92, 85]
}


df = pd.DataFrame(data)
corr_matrix = df.corr()
print(corr_matrix)


plt.figure(figsize=(8, 6))
```

```
sns.heatmap(
    corr_matrix,
    annot=True,            # Show correlation values
    cmap='coolwarm',       # Color palette
    fmt=".2f",             # Format annotation text
    linewidths=0.5,        # Line width between cells
    cbar=True              # Show color bar
)

plt.title("Correlation Heatmap of Subjects", fontsize=14)
plt.show()
```

Q5. Write a Pandas program to merge two DataFrames using different types of joins (inner, outer, left, right). Use sample data representing customer details and order details.

Code:

```
import pandas as pd

customers = pd.DataFrame({
    'CustomerID': [101, 102, 103, 104],
    'CustomerName': ['Ravi', 'Amit', 'Neha', 'Priya'],
    'City': ['Mumbai', 'Delhi', 'Pune', 'Chennai']
})

orders = pd.DataFrame({
    'OrderID': [1001, 1002, 1003, 1004],
    'CustomerID': [101, 103, 105, 102],
    'OrderAmount': [2500, 1800, 3000, 2200]
})
```

```
print("Customer Data:")
print(customers)

print("\nOrder Data:")
print(orders)

inner_join = pd.merge(customers, orders, on='CustomerID', how='inner')
print("\nInner Join Result:")
print(inner_join)

left_join = pd.merge(customers, orders, on='CustomerID', how='left')
print("\nLeft Join Result:")
print(left_join)

right_join = pd.merge(customers, orders, on='CustomerID', how='right')
print("\nRight Join Result:")
print(right_join)

outer_join = pd.merge(customers, orders, on='CustomerID', how='outer')
print("\nOuter Join Result:")
print(outer_join)
```

Q6. Create a Seaborn count plot to visualize the frequency distribution of values in a categorical column from a dataset. Customize colors, orientation, and add value labels on top of bars.

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("tips")
```

```
plt.figure(figsize=(8, 6))
ax = sns.countplot(
    data=df,
    y="day",
    palette="Set2"
)
plt.xlabel("Count")
plt.ylabel("Day")
plt.title("Frequency Distribution of Days")

for p in ax.patches:
    ax.annotate(
        f'{int(p.get_width())}',
        (p.get_width(), p.get_y() + p.get_height() / 2),
        ha='left',
        va='center'
    )

plt.show()
```

Q7. Using NumPy, generate a 1D array of 100 random integers between 1 and 1000. Use Boolean indexing to filter all values greater than 500 and less than 800, and calculate the mean of the filtered values.

Code:

```
import numpy as np

np.random.seed(101)
arr = np.random.randint(1, 1001, 100)
filtered_arr = arr[(arr > 500) & (arr < 800)]
mean_filtered = filtered_arr.mean()
```

```
print("Original Array:")
print(arr)
print("\nFiltered Values (500 < x < 800):")
print(filtered_arr)
print("\nMean of Filtered Values:")
print(mean_filtered)
```

Q8. Load a dataset into Pandas and use Seaborn to plot a histogram showing the distribution of a numerical column. Customize the bin size, color, and add a title.

Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
print(sns.get_dataset_names())
df = sns.load_dataset("tips")
print(df.head())
plt.figure(figsize=(8, 5))

sns.histplot(
    data=df,
    x="total_bill",
    binwidth=5,
    color="skyblue",
    kde=True
)
plt.title("Distribution of Total Bill Amount")
plt.xlabel("Total Bill")
plt.ylabel("Frequency")
plt.show()
```

Q9. Load a dataset and create a Seaborn scatter plot between two numerical columns. Add hue to differentiate categories and customize markers and plot size.

Code:

```
import seaborn as sns

import matplotlib.pyplot as plt

df = sns.load_dataset("iris")

plt.figure(figsize=(8, 6))

sns.scatterplot(

    data=df,

    x="sepal_length",

    y="sepal_width",

    hue="species",

    style="species",

    s=100

)

plt.xlabel("Sepal Length")

plt.ylabel("Sepal Width")

plt.title("Scatter Plot of Sepal Length vs Sepal Width by Species")

plt.show()
```

Q10. Using NumPy, create two matrices (3x3) filled with random integers. Perform matrix addition, subtraction, multiplication, element-wise division, and calculate the determinant and inverse

Code:

```
import numpy as np

np.random.seed(10)

A = np.random.randint(1, 10, size=(3, 3))

B = np.random.randint(1, 10, size=(3, 3))

print("Matrix A:")

print(A)
```

```
print("\nMatrix B:")
print(B)

addition = A + B
print("\nMatrix Addition (A + B):")
print(addition)

subtraction = A - B
print("\nMatrix Subtraction (A - B):")
print(subtraction)

multiplication = np.dot(A, B)
print("\nMatrix Multiplication (A × B):")
print(multiplication)
division = A / B
print("\nElement-wise Division (A / B):")
print(division)
det_A = np.linalg.det(A)
print("\nDeterminant of Matrix A:")
print(det_A)

if det_A != 0:
    inverse_A = np.linalg.inv(A)
    print("\nInverse of Matrix A:")
    print(inverse_A)
else:
    print("\nMatrix A is singular; inverse does not exist.")
```